

2025-2026 春 《数据结构与算法》 第七次作业题解

助教-陈孙一硕

A 根据前序序列重构二叉树

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

int n;
vector<string> seq;
int idx = 0;

TreeNode* build() {
    if (idx >= n || seq[idx] == "#") {
        idx++;
        return nullptr;
    }
    TreeNode* node = new TreeNode(stoi(seq[idx]));
    idx++;
    node->left = build();
    node->right = build();
    return node;
}
```

```

}

void preorder(TreeNode* root) {
    if (!root) return;
    cout << root->val << endl;
    preorder(root->left);
    preorder(root->right);
}

int main() {
    cin >> n;
    seq.resize(n);
    for (int i = 0; i < n; i++) {
        cin >> seq[i];
    }
    TreeNode* root = build();
    preorder(root);
    return 0;
}

```

B 先序和中序构造二叉树

```

#include <iostream>
#include <vector>
using namespace std;

struct TreeNode {
    int val;
    TreeNode* left;
    TreeNode* right;
    TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
};

```

```

TreeNode* build(vector<int>& preorder, vector<int>& inorder, int preL, int preR, int inL, int inR) {
    if (preL > preR) return nullptr;
    int rootVal = preorder[preL];

```

```

TreeNode* root = new TreeNode(rootVal);
int k = inL;
while (k <= inR && inorder[k] != rootVal) k++;
int leftSize = k - inL;
root->left = build(preorder, inorder, preL + 1, preL + leftSize, inL, k - 1);
root->right = build(preorder, inorder, preL + leftSize + 1, preR, k + 1, inR);
return root;
}

void postorder(TreeNode* root) {
    if (!root) return;
    postorder(root->left);
    postorder(root->right);
    cout << root->val << " ";
}

int main() {
    int n;
    cin >> n;
    vector<int> preorder(n), inorder(n);
    for (int i = 0; i < n; i++) cin >> preorder[i];
    for (int i = 0; i < n; i++) cin >> inorder[i];
    TreeNode* root = build(preorder, inorder, 0, n - 1, 0, n - 1);
    postorder(root);
    return 0;
}

```

C 完全二叉树的层序遍历

完全二叉树的编号是按层进行的，那么我们只要将结点编号与结点的值之间构建一个映射即可。

```

//#include<bits/stdc++.h>
#include<iostream>
#include<iomanip>
#include<vector>

```

```

#include<string>
#include<cmath>
#include<algorithm>
#include<set>
#include<map>
#include<unordered_map>
#include<unordered_set>
#include<queue>
#include<stack>
using namespace std;
#define int long long
#define endl '\n'
//const int mod=1e9+7;
const int mod=998244353;
const int N=2e6+101;
int a[N],post[N],mp[N],idx=0,n;
void postorder(int x){
    if (x>n) return;
    postorder(x*2),postorder(x*2+1);
    post[++idx]=x;
}
void solve(){
    cin>>n;
    for (int i=1;i<=n;i++) cin>>a[i];
    postorder(1);
    for (int i=1;i<=n;i++) mp[post[i]]=a[i];
    for (int i=1;i<=n;i++){
        cout<<mp[i];
        if (i==n) cout<<endl;
        else cout<<" ";
    }
}
signed main(){
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    cout.tie(nullptr);

```

```
int T=1; //cin>>T;  
while (T--) solve();  
}
```